

题1① 多场景微架构指标采集

—— 五场景横向对比与差异分析报告
2026 CVM 校企合作考核 · 吴佳豪 · 2026-06

本报告使用 Linux perf stat 工具，对五类典型负载（纯整数计算、浮点矩阵乘、顺序访存、随机访存、分支密集）采集 13 项微架构关键指标，计算衍生指标（IPC / L1 • LLC • TLB Miss Rate / 分支预测失败率），并从 CPU 微架构流水线原理（前端取指/解码、后端执行单元、访存子系统）出发进行差异分析。原始数据见 results/*.txt。

一、测试环境

项	值
CPU	鲲鹏 920 (HiSilicon, TaiShan v110 微架构), aarch64, 4 核 @2.6GHz
缓存	L1d/i 256KiB / L2 2MiB / L3 128MiB (cache line: L1/L2=64B, L3=128B)
NUMA	1 node, 4 核 (0-3) 全在 node 0, 距离矩阵 0→0=10 (单插槽, 无跨节点延迟)
频率策略	performance (钉最高频, 避免动态调频干扰 perf 计数)
虚拟化	KVM guest, perf_event_paranoid = -1 (全开)
OS	openEuler, 内核 4.19.208
工具	perf 4.19.90 (LLC/branch 事件用 raw code)、stress-ng 0.21.03
采集条件	taskset -c 0 钉核 0, 每负载 30s

二、五场景衍生指标对比表

场景	IPC	L1 Miss%	LLC Miss %	分支失败%	dTLB Miss%	内核占比
int64 (ALU 整数)	2.22	0.0016	0.00019	0.031	0.0023	~0
matrixprod (FPU 矩阵)	2.44	5.07	0.0053	0.90	0.0091	~0
read64 (顺序访存)	2.49	0.020	0.0020	0.011	0.013	2.7
rand-set (随机访存)	2.62	0.82	0.36	0.032	0.166	30
queens (分支密集)	1.52	0.0014	0.00015	12.19	0.0013	~0

表 1 五场景微架构衍生指标对比 (加粗为该列极值)

衍生指标计算: IPC = instructions/cycles; L1/LLC/TLB Miss% = 对应 miss 数 / cache-references × 100; 分支失败% = branch-misses / branch-instructions × 100; 内核占比 = sys time / elapsed time。

三、差异分析（结合 CPU 微架构流水线原理）

3.1 IPC 差异：前端→后端流水线视角

$IPC = instructions / cycles$ ，衡量流水线利用度。鲲鹏 920 是超标量乱序核，理论 IPC 可达发射宽度级（4~8）。五场景 IPC 从 1.52（queens）到 2.62（rand-set）跨度显著，根因在“流水线为何空泡（stall）”：

场景	IPC	流水线状态	根因层
queens（1.52 最低）	低	分支预测失败 → 前端取指被冲刷	控制冒险（control hazard）
int64（2.22）	高	ALU 满载，无 stall	理想基准（数据在寄存器）
matrixprod/read64/rand-set	高	后端执行单元 + 访存子系统并行度高	数据冒险但乱序掩盖

关键洞察——IPC 高 ≠ 性能好：rand-set IPC 最高（2.62），但它是最慢的（cache/TLB miss 最多）。原因：cache miss 的 stall 期间周期在走但指令没完成，而 rand-set 计算指令少（主要是访存指令），分母小、IPC 假高。看 IPC 必须结合负载类型，不能单看数值大小。

queens 的控制冒险详解：N-皇后回溯算法产生大量数据相关的条件分支（递归/回溯）。现代深流水线（10+ 级）每次分支预测失败，已取指的后续指令全部作废（冲刷流水线），损失 10+ 周期。queens 分支失败率 12%（每 8 条分支 1 次冲刷），IPC 被严重拖累到 1.52。这是“前端取指被冲刷导致后端执行单元空转”的典型。

3.2 L1 Miss 差异：后端访存子系统——L1 局部性

matrixprod 的 L1 miss 飙到 5.07%（唯一显著场景），根因：矩阵块超过 L1d 容量（256KiB），访问模式“行×列”导致列访问跨 cache line，是经典“cache 不友好”模式（行主序遍历 + 列访问），这正是矩阵乘法优化（分块/tiling）要解决的核心问题。对比 int64/queens（L1 miss ~0.001%）：纯计算负载，数据在寄存器，几乎不访存，故 L1 miss 极低。

3.3 LLC Miss + TLB Miss 差异：后端访存子系统——L3 容量与地址翻译

rand-set 是唯一同时测出 LLC miss + TLB miss + 内核开销三高的场景：

指标	rand-set	其他场景	根因
LLC miss	0.36%（最高）	<0.01%	512M □ L3(128MiB)，随机访问 L3 容量 miss
dTLB miss	0.166%（最高）	<0.02%	512M 随机访问→页表项太多→TLB 容量 miss
内核占比	30%（最高）	<3%	512M 随机写触发内存管理（缺页/TLB refill）

read64 vs rand-set 的教科书对比（同为访存型，访问模式决定 miss 率）：read64（顺序读 1G）L1 miss 0.020%、LLC miss 0.002%——低！硬件预取器识别“顺序 stride”模式，提前把数据从内存拉进 cache；rand-set（随机访问 512M）L1 miss 0.82%、LLC miss 0.36%——高！预取器失效（地址不可预测）。→ 访问模式决定 miss 率，而非数据总量。这是 cache 优化的核心洞察：让访问模式可预测，预取器就能帮你。

3.4 内核占比差异：用户态负载的内核开销

场景	内核占比	根因
int64/matrixprod/queens	~0%	纯用户态计算，无系统调用/缺页
read64	2.7%	1G 分配首次访问触发 page fault
rand-set	30%	512M 随机写触发 munmap/pagemap/TLB refill 等内核内存管理

洞察：访存型负载不仅压 cache/TLB，还压内核内存管理子系统。“用户态负载也可能内核开销大”——随机访存的副作用。

3.5 综合画像：一负载一子系统（设计验证）

负载	设计意图	实测精准压出的瓶颈
int64	压 ALU	IPC 2.22 高 + 全 miss 低（理想基准）
matrixprod	压 FPU	L1 miss 5.07%（最高）
read64	压内存带宽	cache-ref 1490 亿（最高），miss 低（预取器有效）
rand-set	压 TLB/Cache	LLC miss + TLB miss + 内核占比 三高
queens	压分支预测器	IPC 1.52（最低）+ 分支失败 12.19%（最高）

表 2 每种负载都精准压出对应子系统的瓶颈（“一负载一子系统”）

每个负载都精准压出了对应子系统的瓶颈——横向对比时每个指标的异常都能精确归因。

四、工程发现（诚实记录）

4.1 cache-misses ≡ L1-dcache-load-misses（perf 映射冗余）

五场景这两个值完全相同。perf 4.19 在 ARM 上把这两个通用名映射到同一 PMU 事件（L1d refill），二者冗余，分析时取其一即可。

4.2 raw code 事件验证成功

LLC-load-misses（0x037）和 branch-instructions（0x021）均采到真实非零计数——印证 perf 4.19 的 pmu/符号名/ 语法 bug 用 raw code 绕过有效。

4.3 钉核有效性验证

所有场景 cpu-migrations = 0，证明 taskset -c 0 钉核成功，cache 热数据保持、PMU 计数干净，数据可信。

五、结论

- 五场景精准覆盖 CPU 微架构核心子系统（ALU/FPU/内存带宽/TLB-cache/分支预测器），每个负载的指标异常都能精确归因到对应子系统。
- IPC 指标需结合负载类型解读（rand-set IPC 假高揭示了“stall 期间指令未完成”的语义陷阱）。
- 访问模式决定 miss 率（read64 顺序 vs rand-set 随机的巨大差异），是 cache 优化的核心。

- 访存型负载的内核开销不可忽视 (rand-set 内核 30%)，随机访存触发内存管理是主因。
- 工程严谨性：钉核 (migrations=0) + raw code 破解 + 冗余发现，保证数据可信与可复现。